

Отчёт о прохождении учебной практики

Современное программирование, 2 курс
Весенний семестр 2025/2026

Общие сведения

ФИО студента	Пономаренко Никита Михайлович
Название работы	Микросервисная система цифрового обслуживания сети кафе с ИИ-рекомендациями и учётом макронутриентного состава блюд
Ссылка на репозиторий	Не указывается (проект под NDA)
Ментор	Семченко Александр Андреевич Руководитель команды разработки операционной платформы шеринговых сервисов, Яндекс

1. Аннотация

В работе рассматривается задача проектирования и разработки backend-части микросервисного приложения для сети кафе Nutrio. Основное внимание уделено трем сервисам: Sessions для авторизации и управления пользовательскими сессиями, Filter для подбора блюд по пользовательским и профильным ограничениям, а также Recommendations для рекомендаций выбора блюда с использованием LLM. Спроектированы и реализованы gRPC-контракты на Protocol Buffers, PostgreSQL-схемы хранения данных, сервисные слои, репозитории, транспортные обработчики и интеграции между микросервисами. Для сценариев авторизации реализована работа с Yandex OAuth, выпуск JWT access token, хранение refresh token в виде хеша, ротация refresh-сессий и logout через отзыв refresh token. Для фильтрации меню реализован подход построения единого effective filter: параметры запроса объединяются с ограничениями из пользовательского профиля, после чего передаются в Menu service. Для AI-рекомендаций реализован подход, в котором llm отвечает только за рассуждение и выбор, не является источником данных: текущие фильтры и запрос от пользователя сохраняются как асинхронная задача, worker получает блюда-кандидаты через Filter service, передает модели только допустимый набор вариантов и принимает от нее структурированный выбор одного блюда с кратким пояснением. Для устойчивой работы сервиса применены job-worker обработка долгих LLM-вызовов, event-система и SSE для доставки результата клиенту, JSON-контракт ответа модели, validation layer для проверки выбранного блюда, а также repair-loop для повторного запроса при невалидном ответе. В результате получена система рекомендательного подбора блюд, в которой пользователь может сформировать предпочитаемые фильтры, получить список блюд по примененным фильтрам и пояснение выбора наиболее подходящего блюда.

2. Цели и задачи работы

Цель работы: спроектировать и реализовать backend-сервисы авторизации, фильтрации меню и AI-рекомендаций, обеспечив надежное межсервисное взаимодействие, централизованную обработку пользовательской авторизации, профильных ограничений и параметров подбора блюд.

Основные задачи:

1. Спроектировать и реализовать Sessions service для авторизации пользователей через Yandex OAuth, выпуска JWT и управления refresh-сессиями.
2. Спроектировать и реализовать Filter service для построения единого effective filter на основе параметров запроса и ограничений пользовательского профиля.
3. Реализовать межсервисные gRPC-контракты и интеграции между Sessions, Filter, Menu, Recommendations и BDUI gateway.
4. Спроектировать и реализовать Recommendations service для асинхронного выбора наиболее подходящего блюда из списка, полученного по пользовательским фильтрам.
5. Интегрировать Recommendations service с LLM, обеспечив строгий JSON-контракт ответа, валидацию результата, повторную обработку невалидных ответов и доставку рекомендации клиенту через event-систему.

3. Актуальность темы исследования

Сфера общественного питания активно цифровизируется: кафе и рестораны всё чаще используют мобильные приложения, онлайн-заказы, программы лояльности, цифровые меню и рекомендательные механизмы. При этом выбор блюда для пользователя всё ещё может быть неудобным процессом, особенно если меню содержит большое число позиций, а клиент должен учитывать состав блюд, аллергены, индивидуальные пищевые предпочтения и доступность позиций в конкретном филиале. В традиционном сценарии эту задачу частично решает сотрудник кафе, однако такой подход зависит от загруженности персонала, плохо масштабируется и не всегда позволяет учитывать персональные ограничения клиента. Поэтому актуальной задачей является разработка сервиса, который объединяет цифровое меню, фильтрацию блюд, хранение пользовательских предпочтений, ИИ рекомендации и оформление заказа в едином пользовательском сценарии.

Похожие подходы уже развиваются крупными участниками ресторанного рынка. Starbucks использует встроенное приложение в ChatGPT для подбора напитков по настроению, вкусовым предпочтениям, изображению или контексту пользователя с возможностью дальнейшей кастомизации и перехода к оформлению заказа, однако такое решение во многом зависит от внешней платформы. Wendy's FreshAI ориентирован главным образом на автоматизацию приёма и уточнения заказов в drive-thru, а не на полноценную персональную рекомендацию блюд по профилю пользователя. Отдельно существуют решения, реализующие фильтрацию меню по нутриентам, аллергенам и диетическим ограничениям. Однако такие функции часто существуют отдельно друг от друга и не всегда связаны с полным процессом оформления заказа, включая проверку доступности блюд, резервирование позиций и оплату.

В данном проекте эти направления объединяются: реализуется как фильтрация по макронутриентам, аллергенам и другим предпочтениям, так и ИИ рекомендации конкретных блюд по запросу пользователя.

4. Список литературы

1. The Go Programming Language Specification. - URL: <https://go.dev/ref/spec> (дата обращения: 31 марта 2026 г.).
2. PostgreSQL Documentation. - URL: <https://www.postgresql.org/docs/> (дата обращения: 7 апреля 2026 г.).
3. gRPC Documentation. - URL: <https://grpc.io/docs/> (дата обращения: 3 апреля 2026 г.).
4. Yandex Cloud AI Studio Documentation. - URL: <https://yandex.cloud/ru/docs/ai-studio/> (дата обращения: 29 апреля 2026 г.).
5. Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). - IETF, 2015. - URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 23 апреля 2026 г.).

5. Используемый стек технологий

- **Backend-сервисы:** Go, стандартная библиотека Go, gRPC, Protocol Buffers, PostgreSQL, pgx/pgxpool.
- **Авторизация и безопасность:** Yandex OAuth, JWT, хеширование refresh token.
- **Хранение данных и асинхронная обработка:** PostgreSQL, JSONB, pgcrypto, LISTEN/NOTIFY, job-worker обработка фоновых задач.
- **Интеграция сервисов:** gRPC-контракты, Protocol Buffers, BDUI HTTP Gateway, Server-Sent Events (SSE).
- **AI/LLM:** Yandex AI Studio, Yandex Agent API, JSON Schema, mock provider для локального тестирования.
- **Инфраструктура:** Docker, Docker Compose, Makefile.

6. Наиболее значимые коммиты

Хэш	Дата коммита	Краткое описание изменений
-	23.04.2026	Подготовлена финальная версия Filter service: реализовано построение effective filter, объединение параметров запроса с ограничениями профиля и интеграция с Menu service для получения подходящих блюд.
-	06.05.2026	Подготовлена финальная версия Sessions service: реализованы вход через Yandex OAuth, выпуск access token, хранение refresh token, обновление сессий и logout.
-	10.06.2026	Реализована основа Recommendations / LLM service: добавлены gRPC-контракты, PostgreSQL-хранение задач и событий, асинхронная worker-обработка, SSE-доставка результата и интеграция с LLM.
-	24.06.2026	Выполнено упрощение AI-рекомендаций до one-shot сценария: LLM выбирает одно блюдо из backend-подготовленного списка кандидатов, а корректность ответа обеспечивается JSON-контрактом, validation layer и repair-loop.

7. Ключевая инженерная задача

Наиболее значимой инженерной задачей стала разработка надежного сценария AI-рекомендации, в котором LLM помогает пользователю выбрать вариант подходящего блюда. Первоначально рассматривался более сложный диалоговый подход с checkpoint-ами и action-системой: модель должна была поэтапно извлекать фильтры, участвовать в переходах состояния диалога и затем объяснять найденные блюда. Такой вариант оказался сложным для стабилизации, поскольку качество результата зависело одновременно от полноты базы данных, system prompt и способности выбранной модели строго следовать промежуточным интерфейсам взаимодействия. В качестве основного решения был выбран более простой one-shot подход, при котором пользовательские фильтры формируются вручную через Filter service, а LLM работает с уже найденным списком блюд-кандидатов, определяет и комментирует наиболее подходящее пользователю блюдо по его запросу и настройкам фильтрации. Для обработки длительного LLM-вызова применена асинхронная job-worker архитектура: запрос сохраняется как задача, worker получает кандидатов через Filter service, обращается к LLM и публикует результат через event-систему. Для доставки результата клиенту используется SSE, что позволяет не блокировать HTTP-запрос и получать события обработки по мере готовности. Для ограничения недетерминированности LLM применен строгий JSON-контракт ответа, validation layer и repair-loop: backend проверяет, что модель выбрала только существующее блюдо из списка кандидатов, а при ошибке выполняет повторный запрос с указанием причины невалидности. Интеграция с Yandex AI Studio выполнена через backend-подготовленный контекст, поэтому источником истины остаются данные Menu и Filter service, а модель выполняет только выбор и объяснение. В результате получен более устойчивый и демонстрируемый сценарий.

8. Рефлексия о проделанной работе

В ходе работы были развиты навыки проектирования микросервисов на Go, описания gRPC-контрактов через Protocol Buffers, работы с PostgreSQL и построения асинхронных job-worker процессов. Отдельной трудностью стала интеграция LLM: более сложный вариант с системой checkpoint-ов для диалога оказался недостаточно устойчивым, потому что качество результата слишком сильно зависело от доступных моделей LLM, полноты базы данных и system prompt. Для получения воспроизводимого результата было принято решение упростить задачу и использовать модель только для выбора одного блюда из заранее сформированного списка кандидатов. Для корректной реализации исходного диалогового варианта потребовалась бы более мощная LLM-модель с устойчивым следованием системным инструкциям, более полная и нормализованная база данных меню, а также отдельный набор eval-сценариев для проверки переходов между checkpoint-ами. В условиях текущего проекта такие ресурсы были ограничены, поэтому более рациональным решением стало сужение зоны ответственности LLM и перенос критичной логики фильтрации на сторону backend.

9. Описание командной роли

Проект выполнялся в составе команды, разрабатывающей сервис для сети кафе с мобильным клиентом, ИИ рекомендациями и набором backend-сервисов. Другие участники команды занимались клиентским приложением, сервисами меню, заказов, платежей, пользовательского профиля, админской панелью. В рамках практики мой вклад был связан с проектированием и реализацией сервисов Sessions, Filter и Recommendations, отвечающих за авторизацию через Yandex OAuth, построение единого effective filter для фильтрации позиций меню и AI-рекомендацию блюда из предварительно отфильтрованного списка. Разработанные компоненты были интегрированы с остальной системой через gRPC-контракты и BDUI gateway, поэтому результаты моей работы стали частью общего backend-контура приложения. Совокупным результатом командной работы стало приложение для сети кафе, позволяющее пользователю авторизоваться, просматривать меню, учитывать профильные и пользовательские ограничения, получать AI-пояснение выбора блюда и взаимодействовать с сервисами через мобильный интерфейс.

10. Заключение

В ходе практики были спроектированы и реализованы backend-сервисы приложения Nutrio, связанные с авторизацией пользователей, фильтрацией меню и AI-рекомендациями. В Sessions service сформирована логика для авторизованного доступа пользователей и управления сессиями. Filter service обеспечивает механизм применения пользовательских и профильных ограничений к меню. Recommendations service дополняет систему объяснением выбора одного наиболее подходящего блюда из предварительно отфильтрованного списка с использованием LLM. Поставленная цель достигнута: разработанные сервисы интегрированы в общий микросервисный контур и поддерживают ключевые пользовательские сценарии приложения. Полученное решение разделяет детерминированную обработку данных и генеративную часть, оставляя фильтрацию и проверку результатов на стороне backend. Дальнейшее развитие работы может включать расширение тестовых сценариев, улучшение качества данных меню, настройку метрик качества рекомендаций и накопление пользовательской обратной связи по качеству предложения блюд.